
CS-781
CAPSTONE PROJECT
Legal Language Modelling Challenge
BUILDING AI FOR A MORE JUST TOMORROW
LEGAL DATA
LANGUAGE MODELS
LEGAL NLP TASKS
REAL-WORLD IMPACT
UNDERSTAND
REASON
GENERATE
RETRIEVE
ANALYSE
IMPACT
AI FOR ACCESS TO JUSTICE
सत्यमेव जयते
SUPREME COURT OF INDIA
CONSTITUTION
STATUTES
CASE LAW
LEGAL REASONING
PEOPLE
POLICY
TECHNOLOGY
JUSTICE
LANGUAGE. LAW. SOCIETY.
A BRIGHTER, MORE JUST TOMORROW.
* Image generated by ChatGPT

1. Important Instructions **6**
2. Your Assignment: Win the In-House Legal Intelligence System **6**
3. Capstone at a Glance **8**
3.1 Timeline: 8
4. The Journey **9**
5. Dataset and Data Access **10**
5.1 Dataset by phase 10
6. Competition Format **11**
6.1 Competition Link 11
6.2 Registration 11
6.3 There is one submission format across all phases 11
6.4 Use the official test IDs exactly 12
6.5 Submission limits 12
7. Phase 1 - Legal Language Model Pretraining **12**
7.1 What Do You Need to Do? 12
7.2 Data Structure 12
7.2.1 Training Data 13
7.2.2 Development Data 13
7.2.3 Test Data 13
7.2.4 Input 13
7.2.5 Expected Output 14
7.3 Required Architecture Exploration 14
7.4 Experimental Scope 15
7.5 Model and Training Requirements 15
7.6 What You Are Not Allowed to Use 16
7.7 Starter code 16
7.8 Evaluation 17
7.9 What this phase teaches you 17
8. Phase 2 - Legal Named Entity Recognition (L-NER) **17**
8.1 What Do You Need to Do? 17
8.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement 19
8.2.1 Fine-Tuning and Adaptation Techniques 19
8.2.2 Mandatory Knowledge Distillation Experiment 19
8.3 Data Structure 20
8.3.1 Training Data 20
8.3.2 Development Data 20
8.3.3 Test Data 20
8.3.4 Input 21
8.3.5 Expected Output 21
8.4 Evaluation 22

8.5 Expected Analysis..... 22
9. Phase 3 - Rhetorical Role Prediction (RR).....23
9.1 What Do You Need to Do?..... 23
9.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement..... 24
9.2.1 Fine-Tuning and Adaptation Techniques..... 24
9.2.2 Mandatory Knowledge Distillation Experiment..... 25
9.3 Data Structure..... 25
9.3.1 Training Data.....26
9.3.2 Development Data.....26
9.3.3 Test Data.....26
9.3.4 Input..... 26
9.3.5 Expected Output.....27
9.3.6 Rhetorical-Role Labels..... 28
9.4 Evaluation..... 28
9.5 Expected Analysis..... 29
10. Phase 4 - Court Judgment Prediction (CJPE).....29
10.1 What Do You Need to Do?..... 29
10.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement..... 30
10.2.1 Fine-Tuning and Adaptation Techniques..... 31
10.2.2 Mandatory Knowledge Distillation Experiment..... 31
10.3 Data Structure..... 32
10.3.1 Training Data.....32
10.3.2 Development Data.....32
10.3.3 Test Data.....33
10.3.4 Input..... 33
10.3.5 Expected Output.....33
10.4 Evaluation..... 34
10.5 Expected Analysis..... 34
11. Phase 5 - Legal Statute Identification (LSI).....35
11.1 What Do You Need to Do?..... 35
11.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement..... 37
11.2.1 Fine-Tuning and Adaptation Techniques..... 37
11.2.2 Mandatory Knowledge Distillation Experiment..... 38
11.3 Data Structure..... 38
11.3.1 Training Data.....38
11.3.2 Development Data.....38
11.3.3 Test Data.....39
11.3.4 Input..... 39
11.3.5 Expected Output.....39

11.4 Evaluation..... 40
10.5 Expected Analysis..... 40
12. Phase 6 - Abstractive Summarization..... 41
12.1 What Do You Need to Do?..... 41
12.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement..... 42
12.2.1 Fine-Tuning and Adaptation Techniques..... 43
12.2.2 Mandatory Knowledge Distillation Experiment..... 43
12.3 Data Structure..... 43
12.3.1 Training Data..... 44
12.3.2 Development Data..... 44
12.3.3 Test Data..... 44
12.3.4 Input..... 44
12.3.5 Expected Output..... 45
Option A: List Format..... 45
Option B: Object Format..... 45
12.4 Evaluation..... 45
12.5 Expected Analysis..... 45
13. Phase 7 - Prior Case Retrieval (PCR)..... 46
13.1 What Do You Need to Do?..... 46
13.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement..... 47
13.2.1 Fine-Tuning and Adaptation Techniques..... 47
13.2.2 Mandatory Knowledge Distillation Experiment..... 48
13.3 Dataset Structure..... 48
13.3.1 Training Data..... 49
13.3.2 Development Data..... 49
13.3.3 Test Data..... 49
13.3.4 Input..... 49
13.3.5 Expected Output..... 50
13.4 Evaluation..... 51
13.5 Expected Analysis..... 51
14. Phase 8 - TBA..... 52
15. Evaluation and Leaderboards..... 52
15.1 Phase metrics..... 52
15.2 What actually counts for the course..... 52
16. Local Evaluation Before You Submit..... 53
17. Rules, Constraints, and Academic Integrity..... 53
17.1 Individual work..... 53
17.2 Data restrictions..... 53
17.3 Pretrained-model restrictions..... 54

17.4 Compute....54
17.5 LLM and coding-assistant use....54
18. Getting Started....55
18.1 Start with the official resources....55
18.2 A practical workflow....55
Step 1 - Read the data format....55
Step 2 - Build a tiny baseline....55
Step 3 - Scale only after the pipeline works....55
Step 4 - Keep the best checkpoint and the evidence....55
19. Submission Checklist....56
19.1 File and archive....56
19.2 IDs....56
20. Final Report, Code, and Viva....56
20.1 What your report should contain:....57
21. Need Help?....57
22. Closing Note....57
23. References:....58

1. Important Instructions

1. **Read the full competition document carefully before you start. Excuses like you were not aware of a rule will not be entertained.**
2. **This is an individual competition.** Do not share code, model weights, or solutions. Similar submissions can lead to an F grade.
3. **Deadlines are strict and will not be extended** under any circumstances. Do not rely on last-minute Codabench submissions.
4. **Minimum 4 submissions in Phase 1 and a minimum of 6 submissions per phase** (Phases 2–7) is mandatory. Maximum 40 per phase, maximum 10 per day. Identical/repeated submissions do not count toward the minimum.
5. **Submit early in each phase window.** Codabench evaluation takes 15–30 minutes and can be slower under load. Delay excuses will not be entertained.
6. **Username format is mandatory:** firstLetterOfName_Rollno (e.g., a_123456). Other formats will not be registered.
7. **Phases 2–7 must start from your Phase 1 (or later-phase) model.** Other pretrained models and external LLM APIs are strictly prohibited.
8. **Fine-tuning strategy rule (Phases 2–7):** the required 6+ submissions **cannot** be the same method with only different hyperparameters.
9. **Allowed LLM use:** ancillary code (e.g., plots) and understanding architecture. **Not allowed:** writing the whole model architecture, hyperparameter tuning, result analysis/conclusions. Detected misuse → zero + further penalties. The LLM Declaration is mandatory.
10. **Cite papers, tutorials, and external snippets** used in reports and other deliverables.
11. **Ask TAs on Discord (Sanjeet, Shubham, Om) before the window closes**.
12. **NOTE: This is a dynamic document; it will keep getting updated. Please keep checking it regularly.**

2. Your Assignment: Win the In-House Legal Intelligence System

You just got placed at a law firm as an in-house AI developer. It's a good job, and you're proud of it, which is exactly why one of your new colleagues doesn't like it much.

This colleague isn't convinced you're the right person for the work. He's said as much, more than once, in the group chat and in the hallway: he has a freelance AI engineer friend who, in his words, "actually knows how to build this stuff properly." He's gone ahead and told the partners that his friend can build something better than whatever you come up with.

The partners don't want office politics. They want a working system. So they've made a decision: to host a competition on a legal task and make it available to everyone, where each participant will build a system against the same brief, using the same data, under the same constraints, and be judged fairly on the merits. No favoritism, no shortcuts. Just the requirements, the deadlines, and the results.

Here is the brief the firm has given both of you.

The firm's problem is not really about cost; it's about confidentiality. Client matters, privileged communications, and unfiled case material cannot leave the firm's own infrastructure, which rules out sending any of it to a third-party cloud API just to get a language model's help. Large general-purpose language models are powerful, but running client documents through them would mean sending privileged data to someone else's servers, and that's a non-starter for a firm bound by confidentiality obligations. What the partners want instead is a self-hosted, in-house legal language foundation, something that runs entirely on infrastructure they control, never sends client data anywhere, and then prove that this foundation can support a sequence of practical legal NLP capabilities the firm actually needs day to day.

The first challenge is teaching a compact model how legal language works, by training it on the corpus the firm has provided. But learning to continue legal text is only the opening move. A system worth deploying inside a real firm also has to locate the important entities in a document, recognize what role each sentence plays in a legal argument, predict how a case is likely to come out, identify which statutes apply, retrieve a prior case, and compress a long legal document into something a partner can actually read before a meeting.

There will be seven phases. Across seven phases, you will move from building legal language models to information extraction, document structure understanding, judgment prediction, statute identification, prior case retrieval and summarization,. Every phase is a checkpoint where organizers assess system performance using several metrics.

The result should not merely be a single high-scoring run you got lucky with. If this is going into the firm's actual workflow, it needs to be an authentic system, one you understand deeply enough to defend, extend, and fix when it breaks. You are expected to experiment seriously, understand what works and what fails, and be able to explain your design choices under questioning, the same way you'd have to justify a technical decision to a skeptical partner (or, in this case, to a course viva panel standing in for one).

This is an **individual competition**, not a group project. There will be a public leaderboard on Codabench where your solution will be ranked. **Do not share code or models with classmates** unless you want to share grades and career too! Code similarity checks and Viva will be strict: copying leads to **F grade, no questions asked**!

Everyone must participate actively: aim for multiple submissions during the different phases to show your experimentation efforts.

3. Capstone at a Glance

Phase	Task	Input	Output	Primary Metric
1	Legal Language Model Pretraining/ Training	512-word legal prompt	128-word continuation	BertScore
2	Legal Named Entity Recognition	Legal document	Character spans + labels	Strict Macro-F1
3	Rhetorical Role Prediction	Legal document sentences	One role label per sentence	Macro-F1
4	Court Judgment Prediction	Legal case text	Binary prediction (0/1)	Macro-F1
5	Legal Statute Identification	Legal document	One or more statute IDs	Macro-F1
6	Abstractive Summarization	Legal document	Generated legal summary	BERTScore F1
7	Prior Case Retrieval (PCR)	Query legal case + candidate case pool	Top 7 ranked candidate case IDs	Micro-F1@7
8	TBA	TBA	TBA	TBA

All seven Codabench phases are result-submission phases. You submit a prediction.json, packaged in a zip archive, and the scoring program compares your predictions with private ground truth data.

3.1 Timeline:

The current project schedule is:

Phases	**Dates**
Registration on Codabench	26 September, 2026, 12 noon
Phase 1 - Legal Language Model Pretraining	23rd September – 6th October 2026
Phase 2 - Legal Named Entity Recognition	7th – 11th October 2026
Phase 3 - Rhetorical Role Prediction	12th – 16th October 2026

Note: This is a dynamic document and it will keep getting updated, please keep a close watch.

Phases	Dates
Phase 4 - Court Judgment Prediction	17th – 21st October 2026
Phase 5 - Legal Statute Identification	22nd – 26th October 2026
Phase 6 - Abstractive Summarization	27th October – 2nd November 2026
Phase 7: Prior Case Retrieval	3rd – 7th November 2026
Phase 8 - TBA	TBA
Report & Code Submission	9th November 2026 , 11:59 PM IST
Viva	9th – 13th November 2026 (Tentative)

The supplied competition configuration corresponds to the same phase windows when converted from UTC to IST.

NOTE: Please do not rush for last-minute submissions, as Codabench takes around 15–30 minutes to evaluate each submission. Since many students will be participating, the evaluation time may increase due to higher load on Codabench. Therefore, it is advised not to wait until the last moment. Please try to make your submissions in the initial days of the week. Excuses related to Codabench evaluation delays will not be entertained.

Please note: Deadlines will strictly not be extended under any circumstances.

4. The Journey

Phase 1 -Legal Language Model pretraining/ Training : Before a model can do anything sophisticated with legal documents, it needs a useful representation of legal language. You begin by training a compact language model from scratch.

Phase 2 -Legal Named Entity Recognition: Once the system can model legal language, the next challenge is to identify the entities that matter: parties, judges, courts, statutes, precedents, dates, case numbers, and other legal entities.

Phase 3 - Rhetorical Role Prediction: Legal documents are not just bags of facts. Sentences play different rhetorical roles: some state facts, some identify issues, some present arguments, some cite precedents, and some record rulings. Your system must learn that structure.

Phase 4 - Court Judgment Prediction: Now move from structure to decision-oriented prediction. Given legal case text, predict the binary case outcome required by the dataset.

Phase 5 - Legal Statute Identification: A legal decision is rarely complete without identifying the relevant statutory provisions. In this phase, the system must map a legal document to the applicable statute IDs in a multi-label prediction problem.

Phase 6 - Abstractive Summarization: Finally, the system must compress a legal document into an abstractive summary while preserving the information that matters.

Phase 7- Prior Case Retrieval (PCR): Given a legal query or case, the system must retrieve the most relevant prior cases from a collection of legal documents based on their legal and semantic relevance.

Phase 8 - Final Evaluation / Model Submission: Submit your best-performing model for independent evaluation on a task such as Question Answering (QA).

5. Dataset and Data Access

The official dataset is hosted on Hugging Face: https://huggingface.co/datasets/Exploration-Lab/CS-781-Capstone

Evaluation/reference data are not released to participants. The dataset is organized into seven phase-specific configurations:

phase_1 -> Phase_1_Pretraining
phase_2 -> Phase_2_LNER
phase_3 -> Phase_3_RR
phase_4 -> Phase_4_CJPE
phase_5 -> Phase_5_LSI
phase_6 -> Phase_6_Summ
phase_7 -> Phase_7_PCR
phase_8 -> TBA

5.1 Dataset by phase

Phase	Training/development data	Test/reference data
1	Four legal-text training files	Continuation references
2	Two annotated training folds + label list	Fold-3 documents and private annotations
3	Corporate Law and IT train/dev data + label vocabulary	CL/IT test sentences and labels
4	Train/dev case data with binary labels	Test cases and labels
5	Train/dev cases with statute-ID lists	Test cases and labels

Phase	Training/development data	Test/reference data
6	Train documents paired with summaries	Test documents and reference summaries
7	Train/dev query cases with relevant candidate IDs + corresponding candidate case pools	Test query cases + corresponding candidate case pools and private relevance labels
8	TBA	TBA

6. Competition Format

6.1 Competition Link

The competition is hosted on Codabench:
https://www.codabench.org/competitions/18186/?secret_key=5b12ca98-7522-407a-b7e5-7acaac08ffed

6.2 Registration

To take part in the competition, you need to register on Codabench. For this, you need to create an account in the following format: firstLetterOfName_Rollno (e.g., a_123456 for Ashutosh Modi). We suggest you create an account (username in the format above) and register. Make one submission using the starter code to get used to the entire process.
NOTE: If you do not create an account with the above mentioned username format, you will not be registered in the competition.

6.3 There is one submission format across all phases

Every phase uses a file named: prediction.json

Package it as:

```
submission.zip
└── prediction.json
```

The file must be directly at the root of the zip archive. Do not create:

submission.zip
└── some_folder/
  └── prediction.json
Codabench expects the first structure.

6.4 Use the official test IDs exactly

For every phase: include every required official test/example/document ID, do not omit required IDs, do not add unknown IDs. The evaluator checks this explicitly.

6.5 Submission limits

A minimum of 4 valid submissions is mandatory for all phases (Phase 1-7). Students who complete more than the minimum 4 valid submissions in a phase will earn extra points. The maximum number of submissions allowed is 40 per phase, with a per-day maximum of 10 submissions.

Repeated identical submissions will not be considered, whether you have genuinely met the course's minimum-submission requirement. Please do not rush for last-minute submissions, as Codabench takes around 15–30 minutes to evaluate each submission. Since many students will be participating, the evaluation time may increase due to higher load on Codabench. Therefore, it is advised not to wait until the last moment. Please try to make your submissions in the initial days of the week. Excuses related to Codabench evaluation delays will not be entertained.

7. Phase 1 - Legal Language Model Pretraining

7.1 What Do You Need to Do?

In this phase, you will **train a legal language model from scratch** using the provided legal corpus. The objective is to build a foundation model that learns the **vocabulary, syntax, style, and structural patterns of legal text** and to evaluate its ability to model and generate legal language. The model developed in this phase will serve as the **foundation model for Phases 2–7**.

7.2 Data Structure

The Phase-1 dataset consists of four public training sources. The complete Phase-1 corpus contains more than **4.5 billion tokens** across Acts, High Courts, Supreme Court judgments, and Tribunals.

The dataset is organized as follows:

Phase_1_Pretraining/

├── Train/

```
│   ├── train_acts.json
│   ├── train_high_courts.json
│   ├── train_supreme_court.json
│   └── train_tribunals.json
│
└── Test/
    └── Phase1- Test.json
```

7.2.1 Training Data

The Train/ directory contains four legal-text sources:

- train_acts.json — legal Acts and statutory material.
- train_high_courts.json — judgments and legal documents from High Courts.
- train_supreme_court.json — Supreme Court judgments.
- train_tribunals.json — documents from tribunals.

You may use as much of the available training data and token budget as you choose, provided that you remain within the competition rules and your available computational resources.

You should decide how to construct your training corpus from the available sources. You may use all four sources or investigate different combinations and mixing strategies.

7.2.2 Development Data

No separate public development split is provided for Phase 1. You should construct an appropriate validation/development split from the available training data for model selection, checkpoint selection, and experimental analysis.

7.2.3 Test Data

The Test/ directory contains the official test prompts. The test data consists of legal-text prompts for which you must generate continuations.

The corresponding reference continuations are private and are used only for competition evaluation.

7.2.4 Input

The input for each test example is a 512-word legal-text prompt.

```
{
 "1": {
   "text": "The issue involved in these petitions is as to whether ..."
 },
 "2": {
   "text": "In ..."
 }
}
```


7.2.5 Expected Output

The model must generate a 128-word continuation for every test prompt.


```
{
 "1": {
   "text": "Generated 128-word continuation ..."
 },
 "2": {
   "text": "Generated 128-word continuation ..."
 }
}
```


The text field must contain only the generated continuation and must not include the original prompt.

7.3 Required Architecture Exploration

You should use the **BPE tokenizer trained for the course assignment**, or you may train another tokenizer specifically for this phase.

You must explore at least **four substantially different model architectures**. You may select architectures from or be inspired by:

- Decoder-only Transformer
- Encoder-decoder Transformer
- Encoder/hybrid architecture adapted for language modeling
- Efficient or lightweight Transformer architecture
- A new architecture proposed and implemented by you

The four architectures must differ **meaningfully in their architectural design**. Simply changing the hidden size, number of layers, attention heads, learning rate, or other hyperparameters of the same architecture will **not** count as a substantially different architecture.

7.4 Experimental Scope

You should systematically investigate how different design and training choices affect the resulting legal language model. Your experiments may include:

- **Tokenizer design:** BPE or another tokenizer, vocabulary size, and tokenization efficiency.
- **Data selection and mixing:** use of different combinations of Acts, High Courts, Supreme Court judgments, and Tribunals; different sampling or mixing ratios; and the effect of different data sources on model performance.
- **Model architecture:** width, depth, number of attention heads, attention mechanism, feed-forward design, normalization, positional representations, and other architectural components, while remaining within the parameter limit.
- **Context length:** investigate how different context lengths affect language modeling, computational requirements, and the model's ability to capture long-range legal dependencies.
- **Training strategy:** learning-rate schedules, warm-up strategies, batch size, gradient accumulation, optimization settings, and training duration.
- **Regularization:** dropout, weight decay, and other appropriate regularization techniques.
- **Validation strategy:** validation-data construction, checkpoint selection, early stopping, and other methods for selecting a reliable final model.
- **Generation and decoding:** temperature, top-p, top-k, repetition control, and other decoding strategies when evaluating legal text generation.
- **Checkpoint selection:** compare checkpoints based on validation performance and generation quality rather than relying only on the final training checkpoint.
- **Model compression:** investigate knowledge distillation or another suitable compression method to develop a compact version of your trained language model.

You should conduct controlled experiments and analyze how these choices affect both **model quality and computational efficiency**.

7.5 Model and Training Requirements

You must train your language model **from scratch** using the provided legal corpus.

- Your model size must be in between **50M and 300M parameters**.
- You should implement and train your own model architecture rather than relying on a pretrained language model.
- Your training setup should be practical on **free Google Colab/Kaggle-style GPU resources**.
- You may use your own tokenizer or another available tokenizer, subject to the competition rules.
- Non-contextual static embeddings such as **Word2Vec or GloVe** are allowed.
- You may use as much of the provided training corpus and token budget as your computational resources permit.

7.6 What You Are Not Allowed to Use

The following are not allowed in Phase 1:

- **Pretrained language models**
- **Pretrained model weights** from Hugging Face, fairseq, or similar libraries
- **Closed commercial APIs** for training, inference, or evaluation
- Any **foundation model larger than 300M parameters**

The model must therefore learn its language representations from the **provided Phase-1 legal corpus and your chosen architecture/training procedure**.

You should use the results of these experiments to select the **Phase-1 model that will serve as the starting point for fine-tuning and adaptation in Phases 2–7**. (You can select multiple models its upto you)

7.7 Starter code

The provided Phase-1 starter notebook demonstrates one possible route rather than a mandatory architecture. Its current defaults include:

- The Hugging Face dataset Exploration-Lab/CS-781-Capstone
- a GPT-2 tokenizer
- A randomly initialized GPT-2-style language model
- approximately 200M training tokens and 5M validation tokens as starter values
- context/block length 2048
- an 8-layer, 8-head, 560-dimensional model configuration
- generation truncated to 128 words
- generation with sampling parameters exposed for experimentation.

The starter notebook also demonstrates saving the model, generating predictions, writing prediction.json, and creating the Codabench zip archive. **The starter code is provided only for learning purposes and to give participants a basic understanding of the competition, model**

training, submission process, etc. Submitting predictions generated using the starter code will not count toward the minimum submission requirement.

7.8 Evaluation

Primary metric: BERTScore F1

Secondary metrics: ROUGE-L F1, BLEU-4

7.9 What this phase teaches you

Phase 1 is not only about the score. It is your opportunity to study:

- tokenizer choices
- model size versus available compute
- training data selection
- context length
- optimization and learning-rate choices
- decoding strategy
- underfitting/overfitting
- the relationship between validation loss/perplexity and generation quality.

Document your experiments. A weaker model with clear evidence and thoughtful analysis is more valuable than an unexplained lucky submission.

Phase 1 takes place from September 23 to October 6, 2026, with a final deadline of October 6, 2026, 11:59 PM.

8. Phase 2 - Legal Named Entity Recognition (L-NER)

8.1 What Do You Need to Do?

In this phase, you will identify legal named entities in Indian legal documents by predicting their **character-level spans and corresponding entity labels**. You must submit the start character offset, end character offset, and entity label for each predicted entity.

L-NER (Legal Named Entity Recognition) identifies important entities in legal documents and classifies them according to their legal role. Unlike general NER, which focuses on entities such as people, organizations, and locations, L-NER is specifically designed for legal text and identifies entities such as appellants, respondents, counsels, judges, courts, statutes, precedents, witnesses, dates, and case numbers. Your task is therefore to determine **what the entity is, where it occurs in the text, and which legal category it belongs to**.

You must use the **Legal Language Models developed and pretrained by you in Phase 1** as the starting point for this phase. You must experiment with **all four Phase 1 models/architectures** using suitable fine-tuning or adaptation strategies and submit your **best four experiments/submissions** involving different Phase 1 models. Fine-tuning only one model with different techniques will not be sufficient.

You must:

- **Mandatorily use your Phase-1 pretrained Legal Language Models** for this phase.
- Not use any other pretrained language model, pretrained model weights, or external LLM API.
- Fine-tune or adapt your Phase-1 model using the provided L-NER training data.
- Add an appropriate **token-classification, span-classification, or span-extraction head** on top of your Phase-1 model.
- Conduct meaningful experiments with different fine-tuning and parameter-efficient adaptation strategies.
- Investigate appropriate training configurations, including learning rates, batch sizes, sequence lengths, and other hyperparameters.

You should use the phase to investigate factors that directly affect span-level legal NER performance, including:

- **Span extraction versus token-aligned modelling**
- **Document and sentence context windows**
- **Character-to-token and token-to-character alignment**
- **Class-balancing strategies**
- **Task-specific preprocessing**
- **Transfer of representations from the Phase-1 foundation model**
- **Different prediction heads**
- **Different span-decoding strategies**
- **Domain-specific error analysis**

You should compare these approaches based not only on the final F1 score but also on their effect on **exact span boundaries, entity labels, computational efficiency, and model complexity**.

You should pay particular attention to:

- **Character-offset correctness**
- **Document preprocessing**
- **Sentence/token alignment**
- **Span-based versus token-based modelling**
- **Class imbalance**
- **False positives caused by legal boilerplate**
- **Entity boundary errors**
- **Confusion between semantically similar legal entity types**
- **Errors caused by long documents or insufficient context**

A prediction that identifies the correct entity but has an incorrect character boundary is considered incorrect under the strict evaluation protocol.

8.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement

You must **start from and fine-tune/adapt the Legal Language Model developed in Phase 1** for L-NER. You must conduct at least **4 substantially different experiments/submissions** in this phase. Among these:

- At least **3 experiments must use different fine-tuning or parameter-efficient adaptation techniques**. LORMA fine-tuning is compulsory.
- At least **1 experiment must use knowledge distillation**.
- The **knowledge-distillation experiment is mandatory** and counts as one of the required four experiments.
- The four experiments must represent meaningful methodological differences and must not consist only of different learning rates, batch sizes, ranks, sequence lengths, or other hyperparameter changes.

You may conduct additional experiments beyond the required four.

8.2.1 Fine-Tuning and Adaptation Techniques

You should select your required fine-tuning/adaptation experiments from techniques such as:

- Full fine-tuning
- Partial/layer-wise fine-tuning
- Prefix Tuning [4]
- Prompt Tuning [5]
- LoRA [3]
- LORMA [12]
- Houlsby/Pfeiffer Adapters [6,7]
- BitFit or another sparse/parameter-efficient method [8]
- Knowledge Distillation [2]

8.2.2 Mandatory Knowledge Distillation Experiment

You must conduct at least one **knowledge-distillation experiment**.

In this experiment, you should use a larger Phase-1/Phase-2 model as the **teacher** and train a smaller **student model** to transfer useful task-specific knowledge. You should investigate whether the student can retain useful entity-level information while reducing the number of parameters, memory requirements, and computational cost.

8.3 Data Structure

The Phase-2 dataset contains legal documents, entity annotations, and the official entity-label vocabulary.

The dataset is organized as follows:

Phase_2_LNER/

├── Train/

│ ├── fold1-text.json

│ ├── fold1-gold.json

│ ├── fold2-text.json

│ ├── fold2-gold.json

│ └── labels.txt

│

└── Test/

└── fold3-text.json

8.3.1 Training Data

The Train/ directory contains two training folds.

- fold1-text.json and fold2-text.json contain the legal documents and their document IDs.
- fold1-gold.json and fold2-gold.json contain the corresponding entity annotations.
- labels.txt contains the official entity labels that may be predicted.

Each entity annotation specifies the start character offset, end character offset, entity label, and entity text.

8.3.2 Development Data

No separate public development split is provided for Phase 2. You may construct a held-out validation split from the public training data for local model selection and experimentation.

8.3.3 Test Data

The Test/ directory contains:

- fold3-text.json — the test documents and their document IDs.

The entity annotations for the test documents are private and are used only for competition evaluation.

8.3.4 Input

The input consists of legal documents keyed by document ID.

```
{
 "1128835": "REPORTABLE IN THE SUPREME COURT OF INDIA ..."
}
```

Training documents are mapped from document IDs to their complete text. The test set contains the document text without the private annotations.

8.3.5 Expected Output

Predictions must contain character-level entity spans:

```
{
 "1128835": [
   [18, 40, "COURT"],
   [114, 162, "CASENO"],
   [213, 229, "RESP"]
 ]
}
```

Each prediction must contain:

[start_character, end_character, label]

An optional fourth field is accepted:

[start_character, end_character, label, entity_text]

The evaluator scores only the character span and label. The supplied entity_text is ignored for scoring.

The 12 entity labels are:

Label	Meaning
APP	Appellant
RESP	Respondent
A.COUNSEL	Appellant's Counsel
R.COUNSEL	Respondent's Counsel
JUDGE	Judge
WIT	Witness
AUTH	Authority
COURT	Court
STAT	Statute
PREC	Precedent
DATE	Date
CASENO	Case Number

A predicted entity is correct **only when its start offset, end offset, and label exactly match the reference annotation**.

8.4 Evaluation

Primary metric: Strict Macro-F1

Secondary metrics: Strict Macro-Precision, Strict Macro-Recall

Before submitting, you should test your offset-generation and span-decoding code extensively. A prediction that is semantically correct but shifted by even **one character** is considered incorrect under strict span matching. You should therefore validate your complete prediction pipeline, including tokenization, character-to-token alignment, span decoding, offset conversion, label assignment, and output formatting, before making a submission.

8.5 Expected Analysis

Your final analysis should compare the required experiments and discuss:

- Which fine-tuning/adaptation techniques and Phase-1 models perform best under the measured metrics.
- Whether parameter-efficient methods can approach full-model fine-tuning.
- The effect of span-based modelling, context, and preprocessing on L-NER performance.
- Which entity types and span boundaries remain difficult to predict.
- The impact of class imbalance and long legal documents.
- Whether knowledge distillation preserves entity-level performance while improving efficiency.
- How model size, memory, and computational requirements vary across experiments.
- Whether the legal representations learned during Phase 1 transfer effectively to L-NER.

Phase 2 takes place from October 7 to October 11, 2026, with a final deadline of October 11, 2026, 11:59 PM.

9. Phase 3 - Rhetorical Role Prediction (RR)

Rhetorical Role Prediction (RR) is a sentence-level legal NLP task in which you must identify the function or purpose of each sentence within a legal document. For example, a sentence may present facts, state an issue, describe an argument, refer to a statute or precedent, or provide a ruling or reasoning. This task requires the model to understand the structure, context, and function of legal text rather than relying only on individual words.

The dataset covers two legal domains:

- **CL:** Corporate Law
- **IT:** Information Technology

9.1 What Do You Need to Do?

You must use the **Legal Language Models developed and pretrained by you in Phase 1** as the starting point for this phase. You must experiment with **all four Phase 1 models/architectures** using suitable fine-tuning or adaptation strategies and submit your **best four experiments/submissions** involving different Phase 1 models. Fine-tuning only one model with different techniques will not be sufficient.

Your model should take a legal document sentence, together with any context you choose to provide, and predict the corresponding rhetorical-role label.

You must:

- Start from the Legal Language Model developed in **Phase 1**.
- Add a suitable sentence-level classification head or another appropriate prediction mechanism.
- Fine-tune or adapt the Phase 1 model using the provided RR training data.

- Ensure that the model produces exactly one rhetorical-role label for every input sentence.
- Investigate the effect of sentence-level and document-level context on prediction.
- Perform experiments across both the **CL** and **IT** domains.
- Analyze errors involving closely related rhetorical roles and domain-specific differences.

You may experiment with:

- Sentence-only versus context-aware modelling.
- Task-specific preprocessing.
- Class-balancing strategies.
- Prediction heads and sentence representations.
- Transfer of representations learned during Phase 1.
- Domain-specific modelling and error analysis.
- Different fine-tuning and parameter-efficient adaptation strategies.

9.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement

You must conduct **at least 4 substantially different experiments/submissions** in this phase.

These experiments must not consist only of changing hyperparameters of the same method. For example, changing only the learning rate, batch size, LoRA rank, number of epochs, or sequence length does **not** constitute a substantially different experiment.

Among the minimum 4 experiments:

- At least **3 must use different fine-tuning or parameter-efficient adaptation techniques**. LORMA fine-tuning is compulsory.
- At least **1 must be a mandatory knowledge-distillation experiment**.
- The knowledge-distillation experiment counts as **one of the 4 required experiments**.
- You may conduct additional experiments beyond the required 4.
- Hyperparameter variations may be used to optimize each technique, but they do not count as separate techniques.

You must start every experiment from the **Phase 1 Legal Language Model**. You must not use another pretrained language model, pretrained model weights, or an external LLM/API.

9.2.1 Fine-Tuning and Adaptation Techniques

You should select at least 4 distinct techniques from the following approaches and compare their effectiveness for Rhetorical Role Prediction:

- LoRA [3]
- QLoRA [9]
- AdaLoRA [10]

- Rank-Stabilized LoRA (rsLoRA) [11]
- LORMA [12]
- Prefix Tuning [4]
- Prompt Tuning [5]
- IA^3 [13]
- Knowledge Distillation [2]

You should investigate how different low-rank, parameter-efficient, and distillation-based approaches affect the model's ability to distinguish closely related rhetorical roles.

9.2.2 Mandatory Knowledge Distillation Experiment

Knowledge distillation is mandatory in this phase.

You must use a larger or stronger teacher model and transfer its knowledge to a smaller student model. The student should learn from information produced by the teacher, such as logits, probability distributions, intermediate representations, or other task-specific signals.

The distillation experiment must be meaningfully different from the other fine-tuning/adaptation experiments and must be counted as one of the minimum 4 experiments.

9.3 Data Structure

The Phase-3 dataset contains separate training and development data for the **Corporate Law (CL)** and **Information Technology (IT)** domains, together with the official rhetorical-role label vocabulary.

The dataset is organized as follows:

Phase_3_RR/

├── Train/

│ ├── CL_train.json

│ ├── CL_dev.json

│ ├── IT_train.json

│ ├── IT_dev.json

│ └── label_vocab.json

│

└── Test/

├── CL_test.json

└── IT_test.json

9.3.1 Training Data

The Train/ directory contains:

- CL_train.json — Corporate Law training data.
- IT_train.json — Information Technology training data.
- label_vocab.json — the official rhetorical-role labels.

Each training document contains an ordered list of sentences and the corresponding rhetorical-role labels.

9.3.2 Development Data

The development data consists of:

- CL_dev.json — Corporate Law development data.
- IT_dev.json — Information Technology development data.

Each development document contains an ordered list of sentences and their corresponding rhetorical-role labels.

The development labels may be used for model selection, hyperparameter tuning, and local evaluation.

9.3.3 Test Data

The Test/ directory contains:

- CL_test.json — Corporate Law test documents.
- IT_test.json — Information Technology test documents.

The rhetorical-role labels for the test documents are private and are used only for competition evaluation.

9.3.4 Input

Each document contains an ordered list of sentences. For training and development data, each document contains a corresponding ordered list of labels.

For example:

{

"id": "document_001",

```
 "text": [
   "The petitioner filed the present appeal.",
   "The issue before the court concerns the applicability of the statute.",
   "The petitioner argued that the lower court had erred."
 ],
 "labels": [
   "Fact",
   "Issue",
   "ArgumentPetitioner"
 ]
}
```


You may use the sentence itself or additional surrounding document context as input to your model. However, the final prediction must correspond to exactly one rhetorical-role label for each sentence.

9.3.5 Expected Output

Your prediction file must map each document ID to an ordered list of predicted rhetorical-role labels.

The output format must be:


```
{
 "document_001": [
   "Fact",
   "Issue",
   "ArgumentPetitioner",
   "RulingByPresentCourt"
 ]
}
```

The number of predictions must **exactly equal** the number of sentences in the corresponding test document.

For every test document:

len(predictions) == len(test_document["text"])

You must preserve the original sentence order. You must not omit, duplicate, or reorder sentences.

9.3.6 Rhetorical-Role Labels

Every prediction must be one of the following official labels:

Fact

Issue

ArgumentPetitioner

ArgumentRespondent

Statute

Dissent

PrecedentReliedUpon

PrecedentNotReliedUpon

PrecedentOverruled

RulingByLowerCourt

RatioOfTheDecision

RulingByPresentCourt

None

You must use the labels exactly as provided in label_vocab.json. Do not modify the spelling, capitalization, or label names.

9.4 Evaluation

Primary metric: Macro-F1

Secondary metrics: Macro Precision, Macro Recall

9.5 Expected Analysis

Your final analysis should compare the required experiments and discuss:

- Which fine-tuning/adaptation techniques work best for RR under the measured metrics.
- Whether parameter-efficient methods can approach full-model adaptation.
- Whether context improves sentence-level rhetorical-role prediction.
- Which rhetorical roles remain difficult to distinguish.
- Whether performance differs between CL and IT.
- Whether knowledge distillation preserves prediction performance while improving efficiency.
- How model size and computational requirements change across experiments.
- Whether the representations learned during Phase 1 are useful for this task.

Phase 3 takes place from October 12 to October 16, 2026, with a final deadline of October 16, 2026, 11:59 PM.

10. Phase 4 - Court Judgment Prediction (CJPE)

10.1 What Do You Need to Do?

Court Judgment Prediction (CJPE) is a legal NLP task in which you must predict the outcome of a court case from the information contained in the case document. The model is expected to understand relevant facts, legal arguments, reasoning, and other information presented in the case and use this information to predict the final judgment outcome.

In this competition, CJPE is formulated as a **binary classification task**, with two possible outcome labels: 0 and 1 You must use the **Legal Language Model developed in Phase 1** as the starting point for this phase and adapt it for Court Judgment Prediction.

You must use the **Legal Language Models developed and pretrained by you in Phase 1** as the starting point for this phase. You must experiment with **all four Phase 1 models/architectures** using suitable fine-tuning or adaptation strategies and submit your **best four experiments/submissions** involving different Phase 1 models. Fine-tuning only one model with different techniques will not be sufficient.

Your model must:

- Take a complete legal case or an appropriate representation of the case as input.
- Predict the corresponding judgment outcome as either 0 or 1.
- Use a suitable classification head or prediction mechanism.
- Handle the long-document nature of legal judgments appropriately.

- Be trained only using the provided CJPE data and the Phase 1 model.
- Preserve the required output format exactly.

You may experiment with:

- Different methods for representing long legal documents.
- Document chunking and segmentation strategies.
- Different sequence lengths and context windows.
- Hierarchical representations of legal judgments.
- Different pooling strategies.
- Different classification heads.
- Class balancing and class-weighting strategies.
- Task-specific preprocessing.
- Context or section selection.
- Calibration and post-processing.
- Different fine-tuning and parameter-efficient adaptation strategies.

You should investigate whether information from different sections of a legal judgment contributes differently to the final prediction.

Court judgments can contain substantial amounts of text. You should therefore investigate how the model can effectively represent information distributed across a long document.

Possible approaches include:

- Document chunking.
- Overlapping context windows.
- Sliding-window representations.
- Mean or attention-based pooling across chunks.
- Learned document-level pooling.
- Hierarchical sentence/chunk/document representations.
- Section-aware representations.
- Selective context or evidence extraction.
- Classification using representations from different parts of the judgment.

You should compare whether using the entire document, selected sections, or hierarchical representations provides useful information for prediction.

10.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement

You must conduct **at least 4 substantially different experiments/submissions** in this phase.

These experiments must not consist only of changing hyperparameters of the same method. For example, changing only the learning rate, batch size, number of epochs, sequence length, or LoRA rank does **not** constitute a substantially different experiment.

Among the minimum 4 experiments:

- At least **3 must use different fine-tuning or parameter-efficient adaptation techniques**. LORMA fine-tuning is compulsory.
- At least **1 must be a mandatory knowledge-distillation experiment**.
- The knowledge-distillation experiment counts as **one of the 4 required experiments**.
- You may conduct additional experiments beyond the required 4.
- Hyperparameter variations may be used to optimize each technique, but they do not count as separate techniques.

Every experiment must start from the **Phase 1 Legal Language Model**. You must not use another pretrained language model, pretrained model weights or external LLM/API services.

10.2.1 Fine-Tuning and Adaptation Techniques

You should select at least 4 distinct approaches from the following techniques and compare their effectiveness for Court Judgment Prediction:

- Full Fine-tuning
- Partial/Layer-wise Fine-tuning
- Layer Freezing/Unfreezing Strategies
- Bitfit [8]
- LORMA [12]
- AdaLoRA [10]
- Adapters
- Knowledge Distillation [2]

You may also investigate different classification heads and pooling strategies as part of your model design. However, simply changing the classification head or pooling operation while keeping the same adaptation method does not by itself satisfy the requirement for 4 different fine-tuning/adaptation techniques.

10.2.2 Mandatory Knowledge Distillation Experiment

Knowledge distillation is mandatory in this phase. You must use a larger or stronger teacher model and transfer its prediction knowledge to a smaller student model. The student may learn from teacher logits, probability distributions, intermediate representations, or other suitable task-specific signals..

The distillation experiment must be substantially different from the other required adaptation experiments and must count as one of the minimum 4 experiments.

10.3 Data Structure

The Phase-4 dataset contains legal case documents and their corresponding binary judgment outcomes for the training and development splits.

The dataset is organized as follows:

Phase_4_CJPE/

├── Train/

│ ├── train.json

│ └── dev.json

│

└── Test/

└── test.json

10.3.1 Training Data

The train.json file contains training examples with:

- case ID,
- legal case text,
- corresponding binary judgment label.

The preprocessing combines the original single-instance and multi-instance training data into this training split.

10.3.2 Development Data

The dev.json file contains development examples with:

- case ID,
- legal case text,
- corresponding binary judgment label.

The development data may be used for model selection, hyperparameter tuning, and local evaluation.

10.3.3 Test Data

The test.json file contains test cases with:

- case ID,
- legal case text.

The judgment labels for the test cases are private and are used only for competition evaluation.

10.3.4 Input

Each example contains:

```
{
  "id": "case_001",
  "text": "..."
}
```

The training and development data additionally contain the corresponding binary label.

The text field contains the legal case document that you must use for prediction.

You may preprocess or divide the document into suitable chunks or sections internally, but the final prediction must correspond to the original case ID.

10.3.5 Expected Output

You must submit a JSON list of objects in the following format:

```
[
  {
    "id": "case_001",
    "pred": 1
  },
  {
    "id": "case_002",
    "pred": 0
  }
```

]

The pred field must be an **integer** and must be either: 0 or 1.

You must produce exactly one prediction for every test case.

The case IDs in the submission must correspond to the IDs provided in test.json.

10.4 Evaluation

Primary metric: Macro-F1

Secondary metrics: Precision, Recall

Because Macro-F1 is the primary metric, you should pay attention to performance on both outcome classes rather than optimizing only for the majority class.

10.5 Expected Analysis

Your final analysis should compare the required experiments and discuss:

- Which fine-tuning/adaptation techniques are effective for CJPE.
- Whether parameter-efficient methods can retain the performance of full fine-tuning.
- How different long-document representations affect prediction.
- Whether hierarchical or chunk-based representations provide useful information.
- Which sections or contextual information appear useful based on experimental evidence.
- How class imbalance affects performance.
- The characteristics of false-positive and false-negative cases.
- Whether knowledge distillation preserves prediction performance while reducing model size and computational requirements.
- How model size, memory, and inference cost change across experiments.
- How effectively the Phase 1 Legal Language Model transfers to Court Judgment Prediction.

Phase 4 takes place from October 17 to October 21, 2026, with a final deadline of October 21, 2026, 11:59 PM.

11. Phase 5 - Legal Statute Identification (LSI)

11.1 What Do You Need to Do?

Legal Statute Identification (LSI) is the task of identifying which laws or statutory provisions are relevant to a given legal document or case. A statute is a law enacted by a legislature, and legal cases may refer to one or more statutes that are relevant to the facts, arguments, or decision in the case.

In this task, you must identify the corresponding **statute IDs** from the provided set of statutes. Since a legal document can involve multiple statutes, this task is formulated as a **multi-label classification problem**. A single document may therefore have several correct statute IDs.

You must use the **Legal Language Models developed and pretrained by you in Phase 1** as the starting point for this phase. You must experiment with **all four Phase 1 models/architectures** using suitable fine-tuning or adaptation strategies and submit your **best four experiments/submissions** involving different Phase 1 models. Fine-tuning only one model with different techniques will not be sufficient.

Your model must:

- Take a legal document as input.
- Identify all relevant statute IDs for the document.
- Produce a set of statute IDs rather than a single class.
- Use a suitable multi-label classification head or prediction mechanism.
- Be trained using the provided LSI training data.
- Handle long legal documents appropriately.
- Use only the Phase 1 Legal Language Model and the provided competition data.

You may experiment with:

- Different methods for representing long legal documents.
- Multi-label classification heads.
- Different pooling strategies.
- Different loss functions.
- Class-weighting strategies.
- Thresholding strategies for converting prediction scores into statute IDs.
- Sequence lengths and document chunking strategies.
- Context selection and section selection.
- Preprocessing and post-processing methods.
- Calibration techniques.
- Different fine-tuning and parameter-efficient adaptation strategies.

You should investigate whether information learned by the Phase 1 Legal Language Model provides useful representations for identifying statutes from legal documents.

Because a legal document can be associated with multiple statutes, you must treat this task as **multi-label classification** rather than single-label classification.

Your model should be capable of assigning multiple statute IDs to a single document.

You should investigate:

- Binary relevance or independent statute prediction.
- Multi-label classification heads.
- Suitable multi-label loss functions.
- Positive/negative label imbalance.
- Class weighting.
- Rare-statute handling.
- Label co-occurrence patterns.
- Threshold selection.
- Per-label versus global thresholds.
- Calibration of prediction scores.
- Precision/recall trade-offs.

You should ensure that the final prediction contains only valid statute IDs and does not contain duplicate IDs.

Legal documents can contain substantial amounts of text, while the statutes relevant to a case may appear in different sections of the document. You should investigate how to represent long legal documents effectively.

Possible approaches include:

- Document chunking.
- Sliding-window representations.
- Overlapping chunks.
- Chunk-level statute predictions followed by document-level aggregation.
- Mean or attention-based pooling across chunks.
- Hierarchical sentence/chunk/document representations.
- Section-aware representations.
- Targeted context selection.
- Evidence-based or relevance-based context selection.

You should compare whether using the complete document, selected sections, or hierarchical representations provides useful information for identifying relevant statutes.

This is a **multi-label classification task**, not a single-label classification task. A document may have multiple applicable statutes. Therefore, predicting only one statute when several are relevant can reduce

recall even if the predicted statute is correct. You should design your model and decision thresholding strategy accordingly.

11.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement

You must conduct **at least 4 substantially different experiments/submissions** in this phase.

These experiments must not consist only of changing hyperparameters of the same method. For example, changing only the learning rate, batch size, number of epochs, sequence length, or LoRA rank does **not** constitute a substantially different experiment.

Among the minimum 4 experiments:

- At least **3 must use different fine-tuning or parameter-efficient adaptation techniques**. LORMA fine-tuning is compulsory.
- At least **1 must be a mandatory knowledge-distillation experiment**.
- The knowledge-distillation experiment counts as **one of the 4 required experiments**.
- You may conduct additional experiments beyond the required 4.
- Hyperparameter variations may be used to optimize each technique, but they do not count as separate techniques.

You must start every experiment from the **Phase 1 Legal Language Model**. You must not use another pretrained language model, pretrained model weights, or external LLM/API services.

11.2.1 Fine-Tuning and Adaptation Techniques

You should select at least 4 distinct techniques from the following approaches and compare their effectiveness for Legal Statute Identification:

- Full Fine-tuning
- Partial/Layer-wise Fine-tuning
- QLoRA [9]
- LORMA [12]
- AdaLoRA [10]
- Adapters
- Prefix Tuning [4]
- Prompt Tuning [5]
- Knowledge Distillation [2]

You should investigate how different fine-tuning and parameter-efficient adaptation approaches affect multi-label statute identification, particularly for rare statutes and documents containing multiple applicable statutes.

11.2.2 Mandatory Knowledge Distillation Experiment

Knowledge distillation is mandatory in this phase. You must use a larger or stronger teacher model and transfer its knowledge to a smaller student model. The student may learn from teacher logits, probability distributions, intermediate representations, or other suitable task-specific signals.

The distillation experiment must be meaningfully different from the other fine-tuning/adaptation experiments and must be counted as one of the minimum 4 experiments.

11.3 Data Structure

The Phase-5 dataset contains legal case documents and their corresponding applicable statute IDs for the training and development splits.

The dataset is organized as follows:

```
Phase_5_LSI/
├── Train/
│   ├── train.json
│   └── dev.json
│
└── Test/
    └── test.json
```

11.3.1 Training Data

The train.json file contains training examples with:

- case ID,
- legal document text,
- one or more corresponding statute IDs.

Because this is a multi-label classification task, a single document may have multiple applicable statute IDs.

11.3.2 Development Data

The dev.json file contains development examples with:

- case ID,
- legal document text,
- one or more corresponding statute IDs.

The development data may be used for model selection, threshold selection, hyperparameter tuning, and local evaluation.

11.3.3 Test Data

The test.json file contains test documents with:

- case ID,
- legal document text.

The applicable statute IDs for the test documents are private and are used only for competition evaluation.

11.3.4 Input

Training and development examples have the following form:

```
{
  "id": "case_001",
  "text": "...",
  "labels": [1, 7, 23]
}
```

The labels field contains one or more statute IDs relevant to the document.

The test set contains the document ID and text but does not provide the statute labels.

You may internally preprocess, segment, or represent the document using multiple chunks or sections. However, the final prediction must correspond to the original document ID.

11.3.5 Expected Output

You must submit a JSON list of objects in the following format:

```
[
  {
```

```
  "id": "case_001",
  "pred": [1, 7, 23]
 },
 {
  "id": "case_002",
  "pred": [5, 81]
 }
]
```


The pred field must contain:

- **Integers only**.
- **Valid statute IDs** accepted by the competition evaluator.
- **Unique statute IDs within each example**.

You must produce exactly one prediction object for every test document.

The order of statute IDs within pred does not represent a ranking unless otherwise specified by the competition evaluator.

11.4 Evaluation

Primary metric: Macro-F1

Secondary metrics: Micro-F1, Subset Accuracy

The evaluator derives the label dimension from the reference data.

Because Macro-F1 is the primary metric, you should pay particular attention to rare statute IDs and avoid optimizing only for frequently occurring statutes.

10.5 Expected Analysis

Your final analysis should compare the required experiments and discuss:

- Which fine-tuning/adaptation techniques are effective for LSI.
- Whether parameter-efficient methods can retain the performance of full fine-tuning.
- How different long-document representation strategies affect statute identification.

- How rare statutes affect model performance.
- Which loss and class-balancing strategies are useful.
- How threshold-selection strategies affect precision, recall, and Macro-F1.
- Whether multi-label relationships improve prediction.
- Which statute IDs are commonly confused or missed.
- Whether information from particular sections of legal documents is useful.
- Whether knowledge distillation preserves multi-label prediction performance while reducing model size and computational requirements.
- How model size, memory, and inference cost change across experiments.
- Whether the representations learned during Phase 1 are useful for Legal Statute Identification.

Phase 5 takes place from October 22 to October 26, 2026, with a final deadline of October 26, 2026, 11:59 PM.

12. Phase 6 - Abstractive Summarization

12.1 What Do You Need to Do?

Abstractive Legal Summarization is the task of generating a concise summary of a legal document that captures its important information and meaning. Unlike extractive summarization, where the model selects existing sentences or phrases from the original document, abstractive summarization allows the model to generate new sentences in its own words.

For example, given a long court judgment containing the facts of the case, arguments, legal reasoning, and final decision, the model should generate a shorter summary containing the key information.

You must use the **Legal Language Models developed and pretrained by you in Phase 1** as the starting point for this phase. You must experiment with **all four Phase 1 models/architectures** using suitable fine-tuning or adaptation strategies and submit your **best four experiments/submissions** involving different Phase 1 models. Fine-tuning only one model with different techniques will not be sufficient.

Your model must:

- Take a legal document as input.
- Generate a concise abstractive summary of the document.
- Preserve important information and the meaning of the original document.
- Handle long legal documents appropriately.
- Be trained using the provided Phase 6 data.
- Start from the Phase 1 Legal Language Model.

You may experiment with:

- Encoder-decoder or other suitable generation architectures.
- Different fine-tuning and parameter-efficient adaptation strategies.
- Different methods for handling long legal documents.
- Input sequence lengths.
- Document chunking and hierarchical representations.
- Generation length controls.
- Decoding strategies.
- Learning rates, batch sizes, optimizers, and other training hyperparameters.
- Preprocessing and post-processing methods.
- Domain adaptation strategies.
- Redundancy reduction and coverage strategies.

Legal documents can be substantially longer than the input context supported by the model. You should therefore investigate strategies for representing and summarizing long legal documents effectively.

Possible approaches include:

- Document chunking.
- Sliding-window processing.
- Overlapping chunks.
- Hierarchical summarization.
- Chunk-level summaries followed by document-level summarization.
- Section-aware processing.
- Importance-based context selection.
- Hierarchical encoder-decoder representations.
- Combining information from different sections of the document.

You should investigate whether important legal information is lost when documents are divided into chunks and whether hierarchical or selective approaches can improve coverage.

12.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement

You must conduct **at least 4 substantially different experiments/submissions** in this phase.

These experiments must not consist only of changing hyperparameters of the same method. For example, changing only the learning rate, batch size, number of epochs, sequence length, LoRA rank, or generation parameters does **not** constitute a substantially different experiment.

Among the minimum 4 experiments:

- At least **3 must use different fine-tuning or parameter-efficient adaptation techniques**. LORMA fine-tuning is compulsory.
- At least **1 must be a mandatory knowledge-distillation experiment**.
- The knowledge-distillation experiment counts as **one of the 4 required experiments**.

- You may conduct additional experiments beyond the required 4.
- Hyperparameter variations may be used to optimize each technique, but they do not count as separate techniques.

You must start every experiment from the **Phase 1 Legal Language Model**. You must not use another pretrained language model, pretrained model weights or external LLM/API services.

12.2.1 Fine-Tuning and Adaptation Techniques

You should select at least 4 distinct techniques from the following approaches and compare their effectiveness for abstractive legal summarization:

- Houlsby Adapters [6]
- Pfeiffer Adapters [7]
- Prefix Tuning [4]
- Prompt Tuning [5]
- LoRA [3]
- QLoRA [9]
- AdaLoRA [10]
- LORMA [12]
- Knowledge Distillation [2]

You should investigate how different parameter-efficient adaptation methods affect generation quality, factuality, coverage, and computational efficiency.

12.2.2 Mandatory Knowledge Distillation Experiment

Knowledge distillation is mandatory in this phase. You must use a larger or stronger teacher model and transfer its summarization knowledge to a smaller student model. The student may learn from teacher-generated summaries, logits, probability distributions, intermediate representations, or other suitable generation-specific signals.

The distillation experiment must be meaningfully different from the other fine-tuning/adaptation experiments and must be counted as one of the minimum 4 experiments.

12.3 Data Structure

The Phase-6 dataset contains legal documents and their corresponding reference summaries for training.

The dataset is organized as follows:

```
Phase_6_Summ/
├── Train/
│   └── train.json
```

```
└── Test/
    └── test.json
```

12.3.1 Training Data

The train.json file contains training examples with:

- document ID,
- legal document text,
- corresponding reference summary.

Each training example associates a legal document with its reference abstractive summary.

12.3.2 Development Data

No separate public development split is provided for Phase 6. You may construct a held-out validation split from the public training data for model selection and experimentation.

12.3.3 Test Data

The test.json file contains test legal documents and their document IDs.

The corresponding reference summaries are private and are used only for competition evaluation.

12.3.4 Input

Training examples have the following form:

```
{
  "id": "doc_001",
  "document": "...",
  "summary": "Reference summary ..."
}
```

The document field contains the legal document and the summary field contains the reference summary. The test set contains the legal document but does not provide the reference summary.

You may internally preprocess, segment, or represent a document using multiple chunks or sections. However, the final generated summary must correspond to the original document ID.

12.3.5 Expected Output

The evaluator accepts either of the following two formats.

Option A: List Format

```
[
  {
    "id": "doc_001",
    "summary": "Generated legal summary ..."
  }
]
```

Option B: Object Format

```
{
  "doc_001": "Generated legal summary ...",
  "doc_002": "Another generated legal summary ..."
}
```

You should use the **list format** as the straightforward submission format because it explicitly associates each document ID with its generated summary. You must produce exactly one generated summary for every test document.

12.4 Evaluation

Primary metric: BERTScore F1

Secondary metrics: ROUGE-1 F1, ROUGE-L F1

The competition scoring environment computes BERTScore and ROUGE metrics by comparing your submitted summaries with the private reference summaries.

12.5 Expected Analysis

Your final analysis should compare the required experiments and discuss:

- Which fine-tuning/adaptation techniques are effective for legal summarization.
- Whether parameter-efficient methods can retain the quality of full-model adaptation.
- How different long-document strategies affect summarization quality.
- Whether the generated summaries preserve important legal facts and reasoning.
- How well the model covers important information.
- The level of redundancy and repetition in generated summaries.
- How generation length affects BERTScore and ROUGE performance.
- Whether domain adaptation improves legal summarization.

- Whether knowledge distillation preserves summarization quality while reducing model size and computational requirements.
- How model size, memory, and inference cost change across experiments.
- Whether the representations learned during Phase 1 are useful for abstractive legal summarization.

Phase 6 takes place from October 27 to November 2, 2026, with a final deadline of November 2, 2026, 11:59 PM.

13. Phase 7 - Prior Case Retrieval (PCR)

13.1 What Do You Need to Do?

Prior Case Retrieval (PCR) is a legal information retrieval task in which you must retrieve the most relevant prior legal cases for a given query case. The system must understand the legal content of both the query and candidate cases and rank the candidate cases according to their relevance.

You must use the Legal Language Model developed in Phase 1 as the starting point for this phase. You must fine-tune or adapt the Phase 1 model for Prior Case Retrieval. Your model should take a query case and a candidate pool of prior cases and produce a relevance ranking over the candidate cases.

You must:

- Start from the Legal Language Model developed in Phase 1.
- Fine-tune or adapt the Phase 1 model for legal case retrieval.
- Use the provided query and candidate-case documents to learn legal relevance.
- Produce a ranked list of candidate case IDs for every query.
- Retrieve candidates only from the corresponding candidate pool provided for each split.
- Investigate suitable representation, similarity, and ranking strategies for legal case retrieval.
- Analyze how representations learned during Phase 1 transfer to retrieval of legally relevant prior cases.

You may experiment with:

- Different pooling strategies for representing long legal documents.
- Query–candidate similarity functions.
- Contrastive learning objectives.
- Pairwise or listwise ranking objectives.
- Hard-negative and in-batch-negative strategies.
- Sentence-level, section-level, or document-level representations.
- Chunking and hierarchical representations for long cases.
- Different fine-tuning and parameter-efficient adaptation strategies.
- Retrieval efficiency and model-compression techniques.

13.2 Mandatory Fine-Tuning and Knowledge Distillation Requirement

You must conduct at least 4 substantially different experiments/submissions in this phase. These experiments must not consist only of changing hyperparameters of the same method. For example, changing only the learning rate, batch size, LoRA rank, number of epochs, or sequence length does not constitute a substantially different experiment.

Among the minimum 4 experiments:

- At least 3 must use different fine-tuning or parameter-efficient adaptation techniques. LORMA fine-tuning is compulsory.
- At least 1 must be a mandatory knowledge-distillation experiment.
- The knowledge-distillation experiment counts as one of the 4 required experiments.
- You may conduct additional experiments beyond the required 4.
- Hyperparameter variations may be used to optimize each technique, but they do not count as separate techniques.

You must start every experiment from the Phase 1 Legal Language Model. You must not use another pretrained language model, pretrained model weights or external LLM/API services.

13.2.1 Fine-Tuning and Adaptation Techniques

Select at least 4 distinct techniques from the following:

- Full Fine-tuning
- Partial/Layer-wise Fine-tuning
- QLoRA [9]
- LORMA [12]
- Prefix Tuning [4]
- Prompt Tuning [5]
- IA^3 [13]
- Knowledge Distillation [2]

You should investigate how different adaptation strategies affect the ability of the model to represent legal cases and distinguish relevant prior cases from irrelevant candidates. Different pooling strategies, similarity functions, or ranking objectives may be investigated in addition to the required adaptation techniques. These methodological choices alone do not count as separate fine-tuning/adaptation techniques.

13.2.2 Mandatory Knowledge Distillation Experiment

Knowledge distillation is mandatory in this phase. You should use a larger or stronger retrieval model as the teacher and develop a smaller student retrieval model. The student may learn from:

- Teacher similarity scores.
- Teacher probability distributions.
- Pairwise ranking signals.
- Intermediate representations.
- Other task-specific retrieval signals.

The objective is to determine whether a compact retrieval model can preserve retrieval quality while reducing the number of parameters, memory requirements, training cost, and inference cost. The distillation experiment counts as one of the minimum 4 required experiments.

13.3 Dataset Structure

The Phase-7 dataset contains query cases and corresponding candidate-case pools for the training, development, and test splits.

Each query split has its own candidate pool. **The candidate pool corresponding to a split must be used with the queries from that same split.**

The dataset is organized as follows:

Phase_7_PCR/

├── Train/

│ ├── train.json

│ └── train_candidates.json

│

├── Dev/

│ ├── dev.json

│ └── dev_candidates.json

│

└── Test/

 ├── test.json

 └── test_candidates.json

13.3.1 Training Data

The Train/ directory contains:

- train.json — training query cases, their text, and their relevant candidate case IDs.
- train_candidates.json — candidate case documents available for the training queries.

The query and candidate IDs are IndianKanoon case IDs. The text field contains the case document divided into sentences.

13.3.2 Development Data

The Dev/ directory contains:

- dev.json — development query cases, their text, and their relevant candidate case IDs.
- dev_candidates.json — candidate case documents available for the development queries.

The development relevance labels may be used for model selection and local evaluation.

13.3.3 Test Data

The Test/ directory contains:

- test.json — test query cases without their relevance labels.
- test_candidates.json — candidate case documents from which the retrieval results must be selected.

The relevant_candidates field is intentionally removed from the public test queries. The corresponding relevance labels are private and are used only for competition evaluation.

You must retrieve candidates only from the corresponding test_candidates.json candidate pool.

13.3.4 Input

The input consists of a query case together with the corresponding candidate-case pool for that split.

The query case contains its case text in the text field. The text field contains the case document divided into sentences.

For each split, the corresponding candidate pool must be used with that split's queries:

- train.json with train_candidates.json
- dev.json with dev_candidates.json
- test.json with test_candidates.json

For the public test data, the relevant_candidates field is not provided.

13.3.5 Expected Output

Your prediction file must be named prediction.json.

```
{
 "0000021652": [
  "0000590455",
  "0000916840",
  "0001481813",
  "0001907199",
  "0000000987",
  "0000001566",
  "0000002886"
 ],
 "0000030721": [
  "0000004354",
  "0000320809",
  "0000516439",
  "0000611866",
  "0000685234",
  "0000785119",
  "0000849101"
 ]
}
```

For every test query:

- You must provide exactly 7 candidate case IDs.
- The 7 candidate IDs must be unique.
- Candidate IDs must belong to the corresponding test candidate pool.
- Candidates must be ordered from highest to lowest predicted relevance.
- You must provide predictions for every test query.
- Only the top 7 retrieved candidates are evaluated.

13.4 Evaluation

Primary metric: Micro-F1@7

Secondary metrics: Micro-Precision@7, Micro-Recall@7

The gold relevance set may contain fewer or more than 7 relevant candidates. It is neither truncated nor padded. The @7 evaluation considers only the first 7 retrieved candidates for each query and compares them against the complete set of gold relevant candidates.

13.5 Expected Analysis

In your final report, you should compare the required experiments and discuss:

- How different fine-tuning/adaptation techniques affect retrieval quality.
- How Phase 1 legal representations transfer to PCR.
- The effect of pooling and document representation strategies.
- The effect of similarity and ranking objectives.
- The impact of hard-negative and negative-sampling strategies.
- The challenges associated with long legal documents.
- Retrieval errors and common failure patterns.
- The effectiveness of knowledge distillation.
- The trade-off between retrieval quality, model size, memory, and inference efficiency.
- Why a technique that performs well on another legal NLP task may or may not transfer effectively to prior-case retrieval.

Phase 7 takes place from November 3 to November 7, 2026, with a final deadline of November 7, 2026, 11:59 PM.

14. Phase 8 - TBA

An additional 8th phase may also be conducted, in which you may be asked to submit your best-performing model for final evaluation. We will independently evaluate your submitted model on any task, such as the Question Answering (QA) task. Further details regarding the evaluation dataset, format, and procedure, if this phase is conducted, will be shared shortly.

15. Evaluation and Leaderboards

15.1 Phase metrics

Phase	Primary Metric	Secondary Metrics
1	BERTScore F1	ROUGE-L F1, BLEU-4
2	Strict Macro-F1	Strict Macro-Precision, Strict Macro-Recall
3	Macro-F1	Macro-Precision, Macro-Recall
4	Macro-F1	Precision, Recall
5	Macro-F1	Micro-F1, Subset Accuracy
6	BERTScore F1	ROUGE-1 F1, ROUGE-L F1
7	Micro-F1@7	Micro-Precision@7, Micro-Recall@7
8	TBA	TBA

15.2 What actually counts for the course

Leaderboard performance matters, but the course rules explicitly state that it is not the only grading criterion.

The grading criteria include:

- **Your rank in all phases.**
- data analysis and preprocessing
- meeting the minimum submission requirement and demonstrating real experimentation
- code quality and documentation
- analysis of what worked, what failed, and why
- novelty within the compute limits
- Github repo

- viva performance
- final report quality.

16. Local Evaluation Before You Submit

You are strongly encouraged to evaluate locally before submitting to Codabench. The supplied evaluation scripts are designed specifically for this purpose. For Local evaluation, use the provided evaluation script. The local evaluators do not require the organizer-only reference files. Instead, you create your own labelled held-out split and keep its truth/reference answers locally.

This helps you catch:

- wrong JSON structure
- missing IDs
- extra IDs
- invalid labels
- wrong character offsets
- sentence-count mismatches
- invalid statute IDs
- metric implementation mistakes.

17. Rules, Constraints, and Academic Integrity

17.1 Individual work

This is an individual competition. Do not share your code, model weights, or solutions. The course has a strict academic-integrity policy. Substantially similar submissions can lead to an F grade for the students involved.

17.2 Data restrictions

Use only the official competition data.

Strictly prohibited:

- additional judgements
- additional statutes
- scraped court data
- other outside legal datasets.

17.3 Pretrained-model restrictions

For the Phase-1 foundation model:

- Pretrained language models are prohibited
- Pretrained model weights may not be loaded through Hugging Face, fairseq, or another library
- The foundation model must be trained from scratch
- The Phase-1 foundation model must remain within the 50M–300M parameter range.
- Closed commercial APIs such as OpenAI, Gemini, and Claude are prohibited for training, inference, or evaluation.

17.4 Compute

The project is intended to be feasible on free Colab/Kaggle-style GPU resources. It is advised please create multiple accounts on kaggle/Colab. Plan for interruptions and save checkpoints regularly.

17.5 LLM and coding-assistant use

It is OK to use LLM or other sources (websites, blogs, books, etc.) for:

- Coding ancillary parts of the model such as plotting graphs, etc.
- To develop understanding of the model architecture in case it is not clear to you

It is NOT OK to use an LLM or other sources (websites, blogs, books, etc.) for:

- Writing the code for designing the whole model architecture.
- For doing hyperparameter tuning of the model.
- For doing analysis of the results and drawing conclusions about it

It is very straightforward to detect when someone (especially a newbie) is taking code from an LLM or other sources. In case you are found to do so (even for a small part), there will be no questions asked; a direct zero will be awarded to you, along with further penalties which will affect your final grade. In short, you are expected to implement the algorithms **based on course lectures and pseudocode**.
Be a smart agent (remember Honesty is the best Policy); it is ok to lose a few marks rather than getting zero + penalties.
Remember that grading is relative and you are not the only one who is finding it tough!
Papers, tutorials, implementation references, and external snippets used in your work should be cited in the appropriate course deliverables.

18. Getting Started

18.1 Start with the official resources

- Base Paper (IL-TUR: Benchmark for Indian Legal Text Understanding and Reasoning) https://arxiv.org/abs/2407.05399
- Dataset: https://huggingface.co/datasets/Exploration-Lab/CS-781-Capstone
- Starter Code (Phase 1 only): https://drive.google.com/drive/folders/1dZxLOXyWDAW_HmXIHG3NRBzZy4qZeqSI?usp=sharing
- Evaluation Scripts: https://drive.google.com/drive/folders/1cLQw89I1KyBotuY3kgjd1nChs-Msksj4?usp=sharing

18.2 A practical workflow

Step 1 - Read the data format

Before choosing a model, inspect a few real examples from the training data and verify the exact test-input structure.

Step 2 - Build a tiny baseline

Get the entire pipeline working end-to-end before scaling up.

For example:

load data
↓
train a small baseline
↓
generate predictions
↓
run local evaluator
↓
validate JSON

Step 3 - Scale only after the pipeline works

Step 4 - Keep the best checkpoint and the evidence

If you use a post-training approach, document:

- what data or feedback signal you used
- how it was constructed

- how it fits the competition rules
- what changed relative to the base system
- whether the improvement is supported by controlled experiments.

19. Submission Checklist

Before every Codabench submission, verify all of the following.

19.1 File and archive

- ☐ The file is named exactly prediction.json.
- ☐ The archive is a zip file.
- ☐ prediction.json is directly at the root of the archive.
- ☐ There is no extra directory layer.

19.2 IDs

- ☐ Every official test ID is present.
- ☐ No unknown IDs are present.

20. Final Report, Code, and Viva

- **Code:** Create a folder of the best models' code for each phase.
- **GitHub Repository:** Create a well-documented GitHub repository with a proper README and well-documented code for each submission. The README should provide detailed documentation of the architecture, methodology, and implementation for each individual phase and submission. Include as much relevant detail as possible. The GitHub repository link must be included in the final report.
- **Report (LaTeX → PDF):** follow the template (Abstract, Introduction/motivation, methods, experiments, analysis, conclusions).
  - The Report Template is provided here:
  - The LLM declaration will be part of your report, and its questionnaire will be shared in the template.

The report and code submission must be made **on Hello IITK**.

Expected submission archive structure:
CS781-CP-[Firstname]-[Lastname]-[RollNo]-report_code.zip

```
├── code/
│   ├── phase1/
│   ├── phase2/
│   └── ...
└── report/
    └── CS781-CP-[Firstname]-[Lastname]-[RollNo]-report.pdf
```

Note: The **code/** and **report/** folders must be directly at the root of the zip file without any extra outer wrapper directory.

20.1 What your report should contain:

A strong report should make the reader understand:

1. what problem you solved
2. what data you used
3. what baselines you tried
4. what your final approach was
5. what experiments led you there
6. what failed and why
7. what the results show
8. what limitations remain
9. what you would do next with more time or compute.

Do not turn the report into a leaderboard screenshot collection. The important part is the reasoning behind the results.

21. Need Help?

If something is unclear, ask the TAs (Sanjeet, Shubham or Om; on Discord) before submitting rather than after the submission window closes.

22. Closing Note

The point of this capstone was never really about proving a colleague wrong, or beating a freelancer nobody on the team has actually met. The point is to learn what happens when you take a real language problem, work under real resource constraints, build a system, evaluate it honestly, discover where it fails, and then improve it with evidence. If you do that well, the comparison takes care of itself.

Build carefully. Experiment aggressively. Keep records. And make sure you understand every part of the system you submit.

23. References:

1. **Base Paper (IL-TUR: Benchmark for Indian Legal Text Understanding and Reasoning)** **https://arxiv.org/abs/2407.05399**
2. **Knowledge Distillation:** Hinton, G., Vinyals, O., and Dean, J. (2015). *Distilling the Knowledge in a Neural Network.*
   Paper
3. **LoRA:** Hu, E. J., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.*
   Paper
4. **Prefix-Tuning:** Li, X. L. and Liang, P. (2021). *Prefix-Tuning: Optimizing Continuous Prompts for Generation.*
   Paper
5. **Prompt Tuning:**Lester, B., Al-Rfou, R., and Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning.*
   Paper
6. **Houlsby Adapters:** Houlsby, N., et al. (2019). *Parameter-Efficient Transfer Learning for NLP.*
   Paper
7. **Pfeiffer Adapters / AdapterFusion:** Pfeiffer, J., et al. (2021). *AdapterFusion: Non-Destructive Task Composition for Transfer Learning.*
   Paper
8. **BitFit:** Ben Zaken, E., Goldberg, Y., and Ravfogel, S. (2022). *BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models.*
   Paper
9. **QLoRA:** Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs.*
   Paper
10. **AdaLoRA:** Zhang, Q., et al. (2023). *AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning.*
    Paper
11. **Rank-Stabilized LoRA (rsLoRA):** Kalajdzievski, D. (2023). *A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA.*
    Paper
12. **LORMA:** Bihany, B., Patel, A., and Modi, A. (2025). *LORMA.*
    Paper
13. **IA³:** Liu, H., et al. (2022). *Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning.*
    Paper

24. Project FAQs

1. Can we use the transformer from Assignment 3 in the project?
Yes, both versions are allowed:

- The from-scratch decoder-style transformer (Q1-3)
- The `nn.Transformer` / `nn.MultiheadAttention` version (Q5)

You can train either on the legal corpus. The model must have **between 50M and 300M parameters**.

2. Can we add an appendix? 8 pages isn't enough.
Yes, and the appendix has no page limit. All important information must still be in the main report, and anything in the appendix must be referenced from the main report.

3. Can we use datasets other than the one provided?
Yes, at any stage of training. Examples:

- Pre-training: Wikipedia or other generic corpora
- SFT: prompt-response pairs you create
- Alignment: preference data you create

Synthetic data is also allowed. Whatever you use, you must describe your approach in detail in the report.

4. Is the architecture fixed for later phases, or can we change it?
You can change or tweak it, and you can pretrain a new architecture later if needed. Each future phase is only about one week long, so retraining is costly. A strong base model with minor tweaks in later phases is the safer approach.

5. In the architecture exploration part, is it mandatory to make a submission for each type (decoder-only, encoder-decoder, encoder/hybrid, lightweight)?
Yes. You need to make a submission for each architecture type. You can't focus on only 1-2 variants based on BERTScore.

6. My submission is stuck at "Preparing" or "Submitted," or keeps failing.
This is a problem on Codabench's side, not with your code. What to do:

- Retry later. Off-peak hours such as midnight may work better.
- If it stays stuck, delete it and re-upload.
- If you hit the submission limit, tell us and we'll raise it.

7. I'm getting "couldn't connect to huggingface.co" or "roberta-large / BERTScore model not cached."
This is also a Codabench-side issue. Retry after some time. We're working on it.

8. My submission failed. How do I find out why?

Check that your `submission.zip` is in the correct format. If it still fails, post your submission ID in the channel.

9. My submission failed. How do I find out why?

Check that your `submission.zip` is in the correct format. If it still fails, post your submission ID in the channel.

---

25. General Instructions

- **Submit early.** Don't wait for the platform issues to be fixed before you start submitting. Things can still go wrong at the last moment.
- **Post queries in this channel, not in DMs.**
- **Original models only.** If you import a pretrained config and claim you trained it on the provided data, it will not count as an original model and will not count toward your submission.
- **Get free AWS credits.** Apply through the AWS Builder Center student rewards link shared in the channel.
- **Plan your report around the page limit.** 8 pages will not be enough for everything, so plan to use the appendix.
- **Focus on design choices.** The real bottleneck is the design decisions you make at each step, not the coding.
- You also need to add your best submission to the leaderboard else we won't be considering it.
- **Re-upload failed submissions yourself.** They should work on a retry.
- **Use only freely accessible compute resources** (e.g., Kaggle, Colab, AWS Free Tier) to ensure a level playing field. Department-specific resources are strictly prohibited for CS781 assignments and projects. Do not contact CC or attempt to allocate such resources. Violations may result in penalties.